

Fiche Projet — Karaoké collaboratif (GitOps)

1) Contexte

Créer une petite plateforme de **karaoké en salle** : des **salons** (rooms) gérés par un hôte, une **file d'attente** de morceaux alimentée par les participants, des **paroles synchronisées** affichées à l'écran, et un **système de votes** (ex. "skip"). Le tout est livré et exploité via une démarche **GitOps** avec plusieurs environnements (**dev** / **staging** / **prod**).

2) Objectifs pédagogiques

- Travailler en **équipe** avec un vrai **flux Git** (branches, PR, revues).
- Mettre en place un **déploiement automatisé** déclenché par la gestion de versions.
- Gérer **plusieurs environnements** et des **configurations différenciées**.
- Préparer une **démonstration** claire et reproductible.

3) Résultat attendu (fonctionnalités minimales)

- Création/fermeture d'un **salon** avec un **code d'accès**.
- **Catalogue** de morceaux (titres, artistes) fourni par l'équipe.
- **File d'attente** : proposer un morceau, voir l'ordre, retirer/réordonner (par l'hôte).
- **Paroles** (fichiers fournis par l'équipe) affichées de façon **synchronisée**.
- **Votes** simples des participants (ex. "skip") influençant la session.
- **Page de salle** qui se **met à jour en direct** (sans recharger).
- **Tableau simple** de statistiques (ex. chansons jouées, tops).

4) Étapes à réaliser (plan d'action)

Étape A — Cadrage & préparation

1. Rédiger le **backlog** (liste des fonctionnalités) et les **user stories**.
2. Définir les **rôles** (hôte/participants) et les **règles** (ex. seuil de votes "skip").
3. Constituer un **jeu d'essai** : liste de morceaux (métadonnées) + quelques fichiers de paroles (synchronisés par vos soins).
4. Déterminer le **parcours de démo** (scénario public de 3–5 minutes).

Étape B — Maquettes & expérience utilisateur

1. Esquisser les **écrans** :
 - accueil / rejoindre un salon,
 - écran hôte (gestion de la file),
 - écran participants (proposer/voter),
 - écran d'affichage des **paroles**.
2. Définir les **messages d'état** (salon plein, morceau introuvable, vote pris en compte).

Étape C — Organisation GitOps

1. Mettre en place **deux dépôts** :
 - un dépôt **application** (code + doc utilisateur),
 - un dépôt **opérations** (manifeste de déploiement, configs d'environnements).
2. Créer les **trois environnements** : **dev** / **staging** / **prod** avec **configs distinctes** (par ex. tailles max de salon, règles de vote, messages).
3. Définir le **workflow d'équipe** : branches *feature*, **pull requests**, **revues** et **validation**.

4. Paramétrer un **déploiement automatique** sur fusion en branche cible (staging au minimum).

Étape D — Qualité & validation

1. Écrire des **tests de comportement** sur les parcours clés (créer un salon, ajouter un morceau, voter, afficher les paroles).
2. Définir des **vérifications de base** avant déploiement (checklist “prête à déployer”).
3. Préparer un **jeu de données** cohérent pour chaque environnement (ex. moins de morceaux en dev).

Étape E — Démonstration & restitution

1. Organiser une **session de démo** :
 - création d’un salon,
 - participants qui rejoignent,
 - ajout de 2–3 morceaux,
 - affichage des paroles,
 - vote “skip” et passage au suivant.
2. Présenter le **flux GitOps** : montrer une **modification de configuration** (ex. seuil de votes) appliquée automatiquement sur **staging**, puis publiée sur **prod**.
3. Remettre un **dossier de restitution** : guide utilisateur, guide d’exploitation (runbook), et retour d’expérience d’équipe.

5) Organisation & rôles (suggestion)

- **Owner produit** : tient le backlog, tranche sur le périmètre.
- **Référent qualité** : s’assure des tests, de la doc, de la cohérence.
- **Référent déploiement** : pilote la partie GitOps et les environnements.
- **Équipe dev** : prend les user stories, fait des PR courtes et fréquentes.

6) Livrables attendus

- **Backlog** priorisé + **maquettes** d’écrans.
- **Dépôt application** (code, doc utilisateur, jeux d’essai).
- **Dépôt opérations** (organisation des environnements, configurations).
- **Journal de bord** d’équipe (qui a fait quoi, points clés).
- **Guide de démo** (5 min) + **guide d’exploitation** (démarrer, arrêter, dépanner).

7) Critères d’évaluation

- **Fonctionnel** : salons, file d’attente, paroles visibles, votes qui comptent.
- **Collaboration** : branches, PR, revues, résolution de conflits.
- **GitOps** : déploiement automatique à la fusion, séparation claire **app / ops**, configs par environnement.
- **Qualité** : tests de parcours clés, documentation claire, démo fluide.
- **Professionnalisme** : respect des engagements, gestion des priorités, restitution structurée.

8) Extensions (facultatives)

- **Scores** ou mini-jeu (ex. répondre à l’écran pour marquer des points).
- **Mode “soirée”** (file plus longue, seuil de votes différent, réglé par configuration).
- **Page “top titres”** et **historique des salons**.
- **Accessibilité** (contrastes, taille de police pour les paroles).